

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ  
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»

Отчёт по лабораторной работе № 5  
«Сортировки»

Выполнил работу  
Заборщиков Артём  
Академическая группа №J3110  
Принято  
Ментор, Владислав Вершинин

Санкт-Петербург

2024

|                                                      |          |
|------------------------------------------------------|----------|
| <b>Введение</b>                                      | <b>2</b> |
| <b>Подсчёт по памяти</b>                             | <b>2</b> |
| <b>Подсчёт по времени</b>                            | <b>6</b> |
| <b>График зависимости времени от числа элементов</b> | <b>7</b> |

## Введение

Задача - решить задачу сортировки для трёх алгоритмов. Для решения задачи были изучены методы сортировки. Выбраны следующие алгоритмы: OddEvenSort, TreeSort, Сортировка подсчетом.

## Подсчёт по памяти и времени

### OddEvenSort

#### *Память*

$O(1)$  - не требуется дополнительная память, манипуляции проводятся в исходном массиве

#### *Время*

$O(n^2)$  - так как сортировка является модификацией пузырьковой требуется полный проход по массиву столько раз, сколько есть элементов с нарушением порядка.

### TreeSort

#### *Память*

$O(n)$  - создаётся копия массива в виде дерева

#### *Время*

$O(n \log n)$  - в среднем случае строится дерево из  $n$  элементов, стоимость добавления каждого  $\log n$ , т.к. каждый узел в идеальном случае делит массив на 2 равные части, следовательно пройти надо  $\log n$  элементов.

## Сортировка подсчётом

#### *Память*

$O(n + k)$  - требуется создать массив размером с максимальный элемент исходного. Дополнительная копия старого массива не требуется, однако в представленной реализации она есть для удобства.

#### *Время*

$O(n)$  - нам требуется один проход по исходному массиву и один по новому.

## Юнит-тесты

### Методика

В качестве лучшего случая возьмем подмножество натуральных чисел от 1 до  $n$ . Для TreeSort метод будет чуток другой - ему будет передаваться случайный массив, т.к. в случае множества подряд идущих чисел данный метод будет показывать себя наоборот худшим, т.к. одна постоянно растущая ветка. Для среднего случая всем методам передаем случайную последовательность чисел. Худший метод для OddEvenSort будет подмножество натуральных чисел от 1 до  $n$  отсортированное в обратном порядке, для TreeSort аналогично. В случае с сортировкой подсчетом худшего случая не существует (впрочем, как и лучшего), поэтому имеет смысл просто использовать случайный массив. Для регуляризации и

минимизации шума будем делать множественные попытки с разным размером входного массива.

Результаты тестирования:

### TreeSort

| Случай              | Размер<br>массива | Среднее<br>время (с) | Минимальное<br>время (с) | Максимальное<br>время (с) |
|---------------------|-------------------|----------------------|--------------------------|---------------------------|
| Лучший и<br>средний | 100               | 0.00041205           | 0.0002771                | 0.0008032                 |
|                     | 10,000            | 0.0291915            | 0.0268216                | 0.0360419                 |
|                     | 1,000,000         | 0.716488             | 0.658258                 | 0.785766                  |
| Худший              | 100               | 0.00085393           | 0.0004195                | 0.0032777                 |
|                     | 500               | 0.00492439           | 0.0038299                | 0.0069696                 |
|                     | 700               | 0.00781541           | 0.0069234                | 0.0119335                 |

### OddEven

| Случай | Размер<br>массива | Среднее время<br>(с) | Минимальное время<br>(с) | Максимальное время<br>(с) |
|--------|-------------------|----------------------|--------------------------|---------------------------|
|--------|-------------------|----------------------|--------------------------|---------------------------|

|         |           |            |           |           |
|---------|-----------|------------|-----------|-----------|
| Лучший  | 100       | 3.24e-06   | 2.8e-06   | 5.3e-06   |
|         | 10,000    | 0.00059874 | 0.0005455 | 0.0007114 |
|         | 1,000,000 | 0.025702   | 0.0245262 | 0.028729  |
| Средний | 100       | 4.845e-05  | 3.43e-05  | 7.01e-05  |
|         | 10,000    | 0.299285   | 0.142299  | 0.502521  |
| Худший  | 100       | 7.83e-06   | 6.9e-06   | 1.4e-05   |
|         | 1,000     | 6.067e-05  | 5.89e-05  | 6.6e-05   |
|         | 10,000    | 0.00052592 | 0.000264  | 0.0006365 |

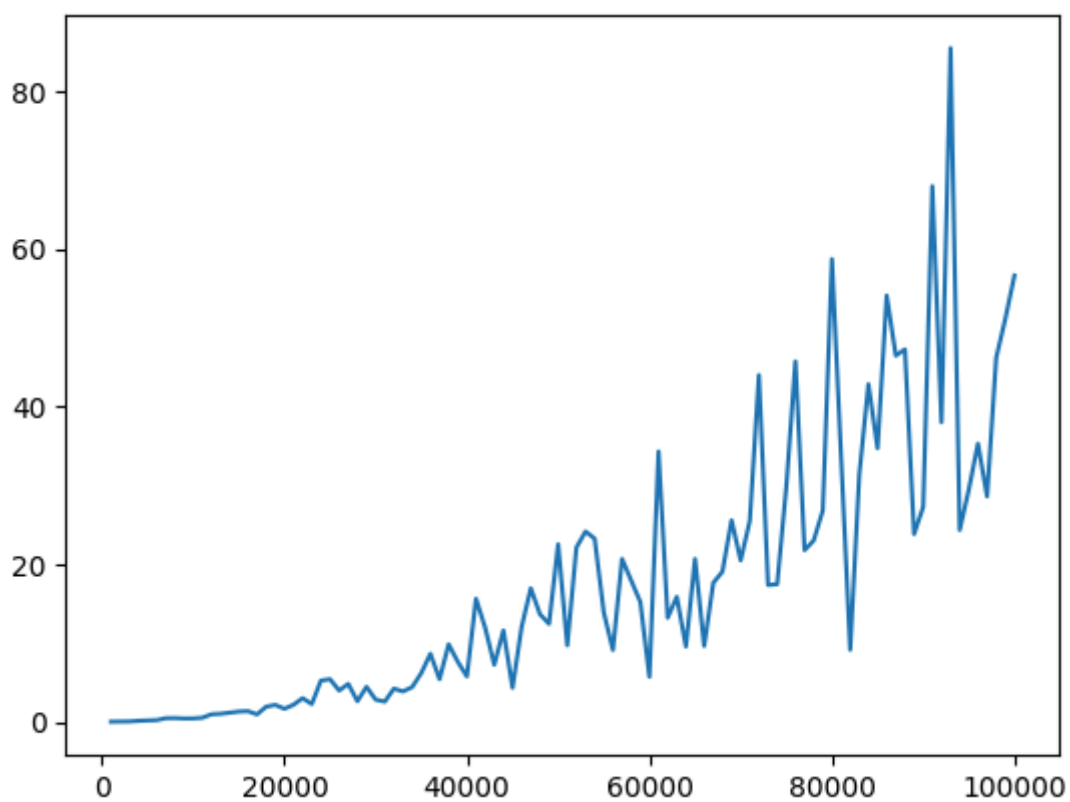
## CountingSort

| Случай | Размер<br>массива | Среднее<br>время (с) | Минимальное<br>время (с) | Максимальное<br>время (с) |
|--------|-------------------|----------------------|--------------------------|---------------------------|
|--------|-------------------|----------------------|--------------------------|---------------------------|

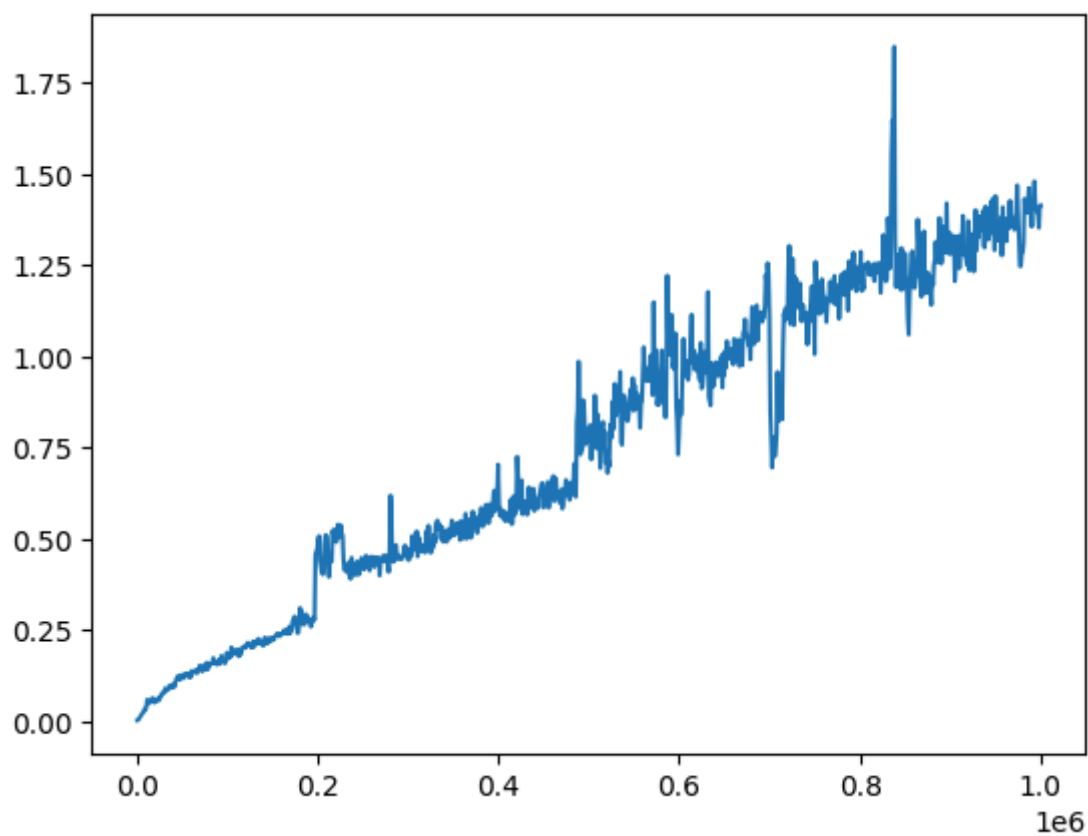
|                  |           |            |           |           |
|------------------|-----------|------------|-----------|-----------|
| Лучший           | 100       | 1.177e-05  | 8.3e-06   | 3.93e-05  |
|                  | 10,000    | 0.00102686 | 0.0009816 | 0.0012116 |
|                  | 1,000,000 | 0.0768903  | 0.0672255 | 0.0888553 |
| Средний и худший | 100       | 0.00045692 | 0.0002569 | 0.0006288 |
|                  | 10,000    | 0.00180004 | 0.0014804 | 0.0022465 |
|                  | 1,000,000 | 0.108411   | 0.0963384 | 0.124331  |

## Зависимость времени выполнения сортировок от количества элементов

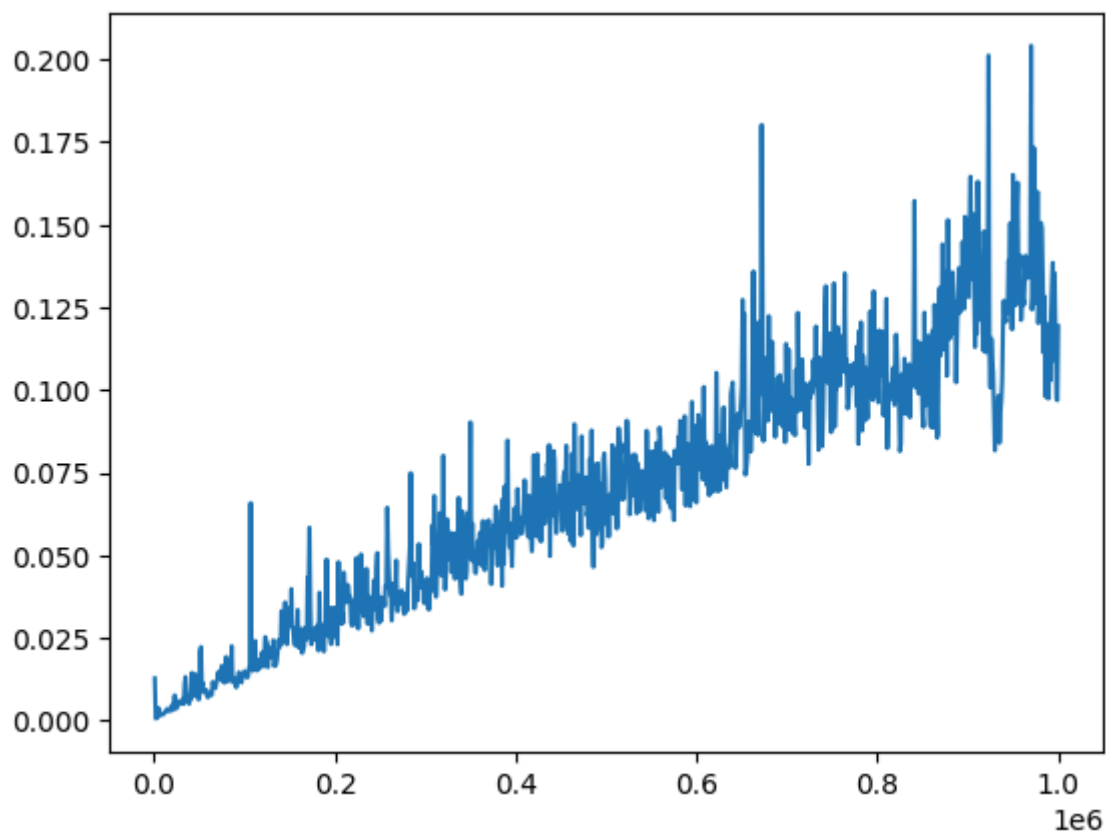
OddEven



TreeSort



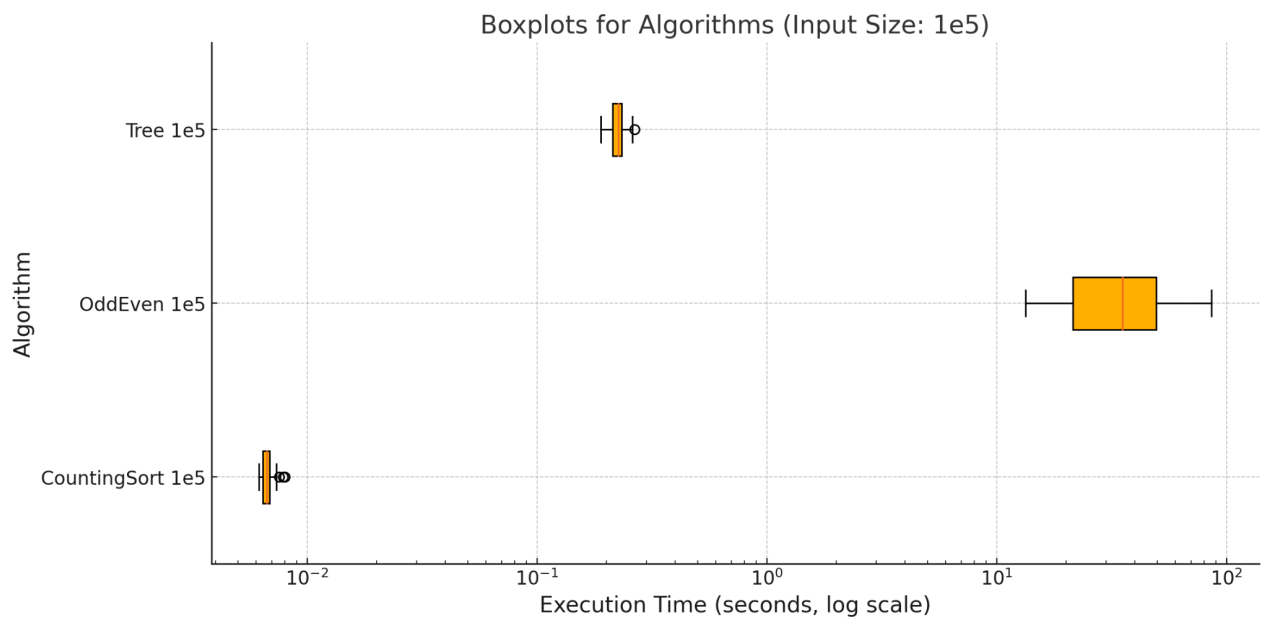
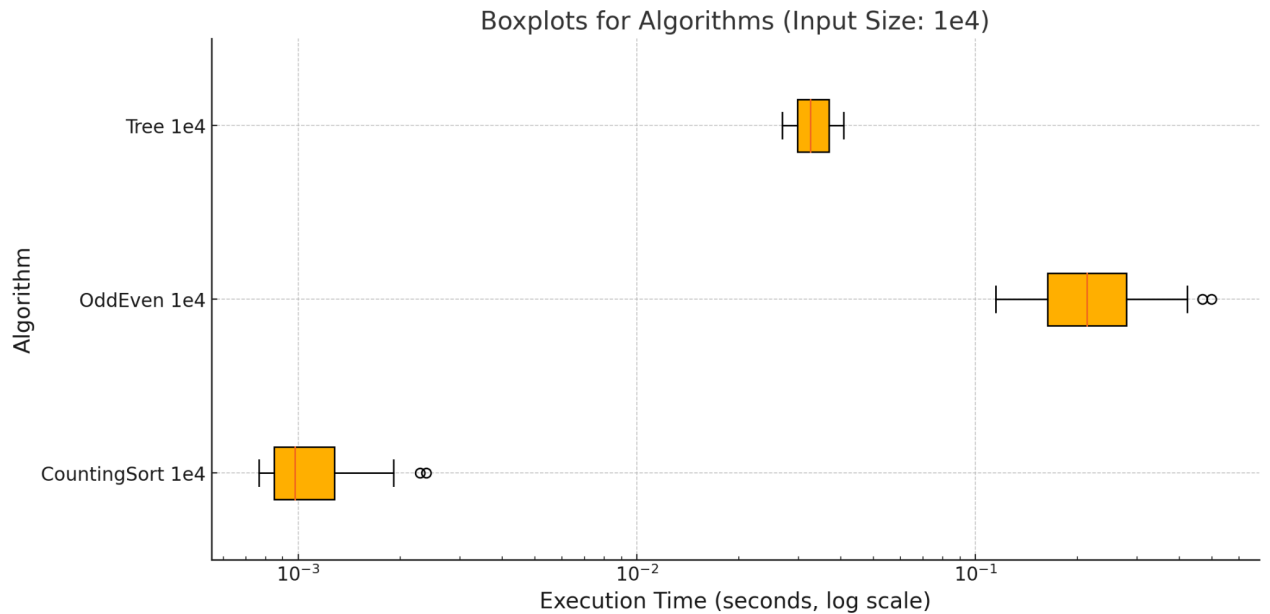
CountingSort





Для OddEven был применён сокращённый эксперимент в связи с запредельным временем. В целом графики соответствуют характеристикам алгоритмов.

## Распределение времени на большом количестве экспериментов



## Вывод

Очевидно напрашивается вывод о том что универсальных алгоритмов сортировки нет – всё сильно зависит от случая и от потребностей. Для целых чисел по времени предпочтителен CountingSort, однако он крайне расточителен по памяти. Идеальным алгоритмом по памяти являются сортировка пузырьком и её модификации, однако она очень долгая. При единичных выбросах она также будет крайне эффективна в том числе и по времени.

## Исходный код программы

```
// sorts_asd.cpp : Этот файл содержит функцию "main". Здесь
// начинается и заканчивается выполнение программы.

#include <iostream>
#include <vector>
#include <cstdlib>
#include <set>
#include <algorithm>
#include <chrono>
#include <numeric>

// я знаю о наличие функции в std, просто хотелось самому сделать
template <typename T>
void swap(T& first, T& second) {
    T t = first;
    first = second;
    second = first;
}

template <typename T>
void OddEvenSort(std::vector<T>& vec) {
    bool issorted = false;
    size_t n = vec.size();

    while (!issorted) {
        issorted = true;
        for (int i = 0; i < n - 1; i += 2) {
            if (vec[i] > vec[i + 1]) {
                swap(vec[i], vec[i + 1]);
                issorted = false;
            }
        }
        for (int i = 1; i < n - 1; i += 2) {
            if (vec[i] > vec[i + 1]) {
                swap(vec[i], vec[i + 1]);
                issorted = false;
            }
        }
    }
}
```

```

}

template<typename T>
struct Node {
    T value;
    unsigned int cnt;

    ~Node();
    Node(T value);
    void AddNode(const T& value);    // Добавление нового узла
    const std::vector<T> GetSortedVector();
    Node(const std::vector<T>& vec);

private:
    Node* LEFT;
    Node* RIGHT;
};

template<typename T>
Node<T>::~~Node() {
    delete LEFT;
    delete RIGHT;
}

template<typename T>
Node<T>::Node(T val) : value(val), cnt(1), LEFT(nullptr),
RIGHT(nullptr) {}

template<typename T>
void Node<T>::AddNode(const T& val) {
    if (val == value) {
        ++cnt;
    }
    else if (val < value) {
        if (LEFT) {
            LEFT->AddNode(val);
        }
        else {
            LEFT = new Node<T>(val);
        }
    }
    else { // val > value
        if (RIGHT) {

```

```

        RIGHT->AddNode(val);
    }
    else {
        RIGHT = new Node<T>(val);
    }
}
}

template<typename T>
Node<T>::Node(const std::vector<T>& vec) : value(vec[0]), cnt(1),
LEFT(nullptr), RIGHT(nullptr) { //
    for (size_t i = 1; i < vec.size(); ++i) {
        AddNode(vec[i]);
    }
}

template<typename T>
const std::vector<T> Node<T>::GetSortedVector() {
    std::vector<T> out;
    if (LEFT) {
        std::vector<T> leftLeaf = LEFT->GetSortedVector();
        out.insert(out.end(), leftLeaf.begin(), leftLeaf.end());
    }

    int temp_cnt = cnt + 1; // так как --i возвращает rvalue требуется
доп итерация
    while (--temp_cnt)
        out.push_back(value);

    if (RIGHT) {
        std::vector<T> rightLeaf = RIGHT->GetSortedVector();
        out.insert(out.end(), rightLeaf.begin(), rightLeaf.end());
    }

    return out;
}

std::vector<unsigned int> generateintvec(
    const size_t& n,
    const unsigned int& max)
{
    std::vector<unsigned int> vec(n);

```

```

    for (int i = 0; i < n; i++)
        vec[i] = rand() % (max + 1);

    return vec;
}

template <typename T>
bool checkOrder(const std::vector<T>& vec) {
    for (int i = 1; i < vec.size(); ++i)
        if (vec[i - 1] > vec[i])
            return false;

    return true;
}

//template <typename T>
//std::vector<T> multisetsort(const std::vector<T>& vec) {
//    std::set<T> set(vec.begin(), vec.end());
//    std::vector<T> newvec = std::vector<T>(set.begin(), set.end());
//    return newvec;
//}

template <typename T>
std::vector<T> countingSort(const std::vector<T>& vec) {
    const T& max_elem = *max_element(vec.begin(), vec.end());
    std::vector<size_t> count_vec(max_elem + 1, 0);

    for (const T& elem : vec)
        ++count_vec[elem];

    std::vector<T> out(vec.size());
    int j = 0;
    for (size_t i = 0; i <= max_elem; ++i)
        while (count_vec[i] != 0) {
            out[j] = i;
            ++j;
            --count_vec[i];
        }

    return out;
}

```

```

}

void check(std::vector<bool> preds, std::vector<bool> truth) {
    if (preds.size() != truth.size()) {
        std::cout << "Can't do tests, becuse sizes of sets are not
equal. "
                << "Size of preds: " << preds.size()
                << " elems, size of truth: " << truth.size()
                << " elems\n";
        return;
    }

    size_t n = truth.size();
    size_t cnt = n;

    for (int i = 0; i < n; i++)
        if (preds[i] != truth[i] && cnt--)
            std::cout << "Error in test #" << i + 1 << '\n';

    std::cout << "Done " << cnt << " tests from " << n << '\n';
}

const void unit_tests() {
    std::vector<bool> checkint = { true, false, true , false };
    std::vector<std::vector<int>> testdata =
    { {1, 2, 4, 5},
      {1, 2, 5, 4},
      {1, 1, 1},
      {2, 1, 1}
    };

    std::vector<bool> checkdouble = { true, false, true, false };
    std::vector<std::vector<double>> testdata_double =
    { {1, 2, 4.5, 5},
      {1, 2, 4.5, 4},
      {1, 1, 1.00001},
      {2, 1.1, 1.01}
    };

    std::vector<bool> checkgen = { false, false, false, true };

    std::cout << "Order checker on int\n";
}

```

```

std::vector<bool> preds;
for (const std::vector<int>& vec : testdata)
    preds.push_back(checkOrder(vec));
check(preds, checkint);

std::cout << "\nOrder checker on double\n";
std::vector<bool> preds2;
for (const std::vector<double>& vec : testdata_double)
    preds2.push_back(checkOrder(vec));
check(preds2, checkdouble);

std::cout << "\nVector generator\n\n";
std::vector<bool> preds3;
for (const int& n : { 2342, 2334, 1314, 1 })
    preds3.push_back(checkOrder(generateintvec(n, 100)));
check(preds3, checkgen);

const int attempts = 10;
std::cout << "TreeSort tests\n---\nBest&Mean case\n\n";
for (const int& n : { 100, 10000, 1000000 }) {
    std::vector<double> t(attempts);
    for (int i = 0; i < attempts; i++) {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(n,
2000000);
        Node<unsigned int> tree(vec);
        if (!checkOrder(tree.GetSortedVector())) throw;

        auto end = std::chrono::high_resolution_clock::now();
        t[i] = (end - start).count()/1e9;
    }

    double mean = 0;
    for (const double& el : t) mean += el;
    mean /= attempts;
    std::cout << "Array Size=" << n
        << "\nMean time=" << mean
        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
<< "\n\n";
}

std::cout << "Worst case\n\n";

```

```

for (const int& n : { 100, 500, 700 }) {
    std::vector<double> t(attempts);
    for (int i = 0; i < attempts; i++) {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec (n);
        for (int i = n - 1; i != -1; i--) vec[i] = i;
        Node<unsigned int> tree(vec);
        if (!checkOrder(tree.GetSortedVector())) throw;
        auto end = std::chrono::high_resolution_clock::now();
        t[i] = (end - start).count() / 1e9;
    }

    double mean = 0;
    for (const double& el : t) mean += el;
    mean /= attempts;
    std::cout << "Array Size=" << n
        << "\nMean time=" << mean
        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
    << "\n\n";
}

std::cout << "OddEven tests\n---\nBest case\n\n";
for (const int& n : { 100, 10000, 1000000 }) {
    std::vector<double> t(attempts);
    for (int i = 0; i < attempts; i++) {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec(n);
        for (int i = 0; i < n; i++) vec[i] = i;
        OddEvenSort(vec);
        if (!checkOrder(vec)) throw;
        auto end = std::chrono::high_resolution_clock::now();
        t[i] = (end - start).count() / 1e9;
    }

    double mean = 0;
    for (const double& el : t) mean += el;
    mean /= attempts;
    std::cout << "Array Size=" << n
        << "\nMean time=" << mean
        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
    << "\n\n";
}

```



```

}

std::cout << "Mean case\n\n";
for (const int& n : { 100, 10000 }) {
    std::vector<double> t(attempts);
    for (int i = 0; i < attempts; i++) {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(n,
2000000);
        OddEvenSort(vec);
        if (!checkOrder(vec)) throw;
        auto end = std::chrono::high_resolution_clock::now();
        t[i] = (end - start).count() / 1e9;
    }

    double mean = 0;
    for (const double& el : t) mean += el;
    mean /= attempts;
    std::cout << "Array Size=" << n
        << "\nMean time=" << mean
        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
<< "\n\n";
}

std::cout << "Worst case\n\n";
for (const int& n : { 100, 1000, 10000 }) {
    std::vector<double> t(attempts);
    for (int i = 0; i < attempts; i++) {
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec(n);
        for (int i = n - 1; i != -1; i--) vec[i] = i;
        OddEvenSort(vec);
        if (!checkOrder(vec)) throw;
        auto end = std::chrono::high_resolution_clock::now();
        t[i] = (end - start).count() / 1e9;
    }

    double mean = 0;
    for (const double& el : t) mean += el;
    mean /= attempts;
    std::cout << "Array Size=" << n
        << "\nMean time=" << mean

```

```

        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
<< "\n\n";
    }

    std::cout << "CountingSort tests\n---\nBest case\n\n";
    for (const int& n : { 100, 10000, 1000000 }) {
        std::vector<double> t(attempts);
        for (int i = 0; i < attempts; i++) {
            auto start = std::chrono::high_resolution_clock::now();
            std::vector<unsigned int> vec(n);
            for (int i = 0; i < n; i++) vec[i] = i;
            if (!checkOrder(countingSort(vec))) throw;
            auto end = std::chrono::high_resolution_clock::now();
            t[i] = (end - start).count() / 1e9;
        }

        double mean = 0;
        for (const double& el : t) mean += el;
        mean /= attempts;
        std::cout << "Array Size=" << n
            << "\nMean time=" << mean
            << "\nMin time=" << *std::min_element(t.begin(), t.end())
            << "\nMax time=" << *std::max_element(t.begin(), t.end())
<< "\n\n";
    }

    std::cout << "Mean&Worst case\n\n";
    for (const int& n : { 100, 10000, 1000000 }) {
        std::vector<double> t(attempts);
        for (int i = 0; i < attempts; i++) {
            auto start = std::chrono::high_resolution_clock::now();
            std::vector<unsigned int> vec = generateintvec(n,
2000000);
            if (!checkOrder(countingSort(vec))) throw;
            auto end = std::chrono::high_resolution_clock::now();
            t[i] = (end - start).count() / 1e9;
        }

        double mean = 0;
        for (const double& el : t) mean += el;
        mean /= attempts;
        std::cout << "Array Size=" << n

```

```

        << "\nMean time=" << mean
        << "\nMin time=" << *std::min_element(t.begin(), t.end())
        << "\nMax time=" << *std::max_element(t.begin(), t.end())
    << "\n\n";
    }
}

const void speedtest() {
    std::vector<double> t1;
    std::cout << "\nTreeSort\n---\n";
    for (int i = 1000; i <= 1000000; i += 1000) {
        std::cout << '\r' << static_cast<double>(i) / 10000.0 << "%,
i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(i, 2000000);
        Node<unsigned int> tree(vec);
        auto vec2 = tree.GetSortedVector();
        auto end = std::chrono::high_resolution_clock::now();
        t1.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t1) std::cout << el << " \n";

    std::vector<double> t2;
    std::cout << "\nCountingSort\n---\n";
    for (int i = 1000; i <= 1000000; i += 1000) {
        std::cout << '\r' << static_cast<double>(i) / 10000.0 << "%,
i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(i, 2000000);
        auto vec2 = countingSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t2.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t2) std::cout << el << " \n";

    std::vector<double> t;
    std::cout << "\n SORT SPEED\n";
    std::cout << "\nOddEvenSort\n---\n";
    for (int i = 1000; i <= 100000; i += 1000) {
        std::cout << '\r' << static_cast<double>(i) / 1000.0 << "%,
i=" << i;

```

```

        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(i, 2000000);
        OddEvenSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t) std::cout << el << " \n";
}

const void boxplot_data_counting() {
    std::vector<double> t;
    std::cout << "\nCountingSort 1e4\n---\n";
    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << static_cast<double>(i) * 2 << "%, i=" <<
i;

        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(10000, 10000);
        auto vec2 = countingSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t) std::cout << el << " \n";

    std::vector<double> t2;
    std::cout << "\nCountingSort 1e5\n---\n";
    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << i * 2 << "%, i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(100000,
100000);
        auto vec2 = countingSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t2.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t2) std::cout << el << " \n";
}

const void boxplot_data_bubble () {
    std::vector<double> t;
    std::cout << "\nOddEven 1e4\n---\n";

```

```

    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << i * 2 << "%, i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(10000, 10000);
        OddEvenSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t) std::cout << el << " \n";

    std::vector<double> t2;
    std::cout << "\nOddEven 1e5\n---\n";
    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << i * 2 << "%, i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(100000,
100000);
        OddEvenSort(vec);
        auto end = std::chrono::high_resolution_clock::now();
        t2.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t2) std::cout << el << " \n";
}

const void boxplot_data_tree() {
    std::vector<double> t;
    std::cout << "\nTree 1e4\n---\n";
    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << i * 2 << "%, i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(10000, 10000);
        Node<unsigned int> tree(vec);
        auto vec2 = tree.GetSortedVector();
        auto end = std::chrono::high_resolution_clock::now();
        t.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t) std::cout << el << " \n";

    std::vector<double> t2;
    std::cout << "\nTree 1e5\n---\n";

```

```

    for (int i = 0; i < 50; i++) {
        std::cout << '\r' << i * 2 << "%, i=" << i;
        auto start = std::chrono::high_resolution_clock::now();
        std::vector<unsigned int> vec = generateintvec(100000,
100000);
        Node<unsigned int> tree(vec);
        auto vec2 = tree.GetSortedVector();
        auto end = std::chrono::high_resolution_clock::now();
        t2.push_back((end - start).count() / 1e9);
    }
    std::cout << '\n';
    for (const double& el : t2) std::cout << el << " \n";
}

int main()
{
    boxplot_data_counting();
    boxplot_data_bubble();
    boxplot_data_tree();
    unit_tests();
    speedtest();
    return 0;
}

```